BilinearInterpolation

Bilinear interpolation are composed of two parts:

1. Grid generator.
2. Grid sampler/interpolator.

```
# -*- coding: utf-8 -*-
"""
Created on Mon Jan 19 12:54:02 2026

@author: rslim
"""

import cv2
import math
import numpy as np
import torch
import torch.nn.functional as F

def linear_resize(in_array, size):
    """
    Parameters
    ----------
    in_array : input 1D array.
    size : desired size

    Returns
    -------
    resized array

    """
    ratio = (len(in_array) - 1) / (size - 1)
    out_array = []

    for i in range(size):
        low = math.floor(ratio * i)
        high = math.ceil(ratio * i)
        weight = ratio * i - low

        a = in_array[low]
        b = in_array[high]

        out_array.append(a * (1 - weight) + b * weight)
    return np.array(out_array)

def bilinear_resize(image, height, width):
    """
    Parameters
    ----------
    image : 2D numpy array.
    height : desired height
    width : desired width

    Returns
    -------
    resized image
```

```
    """
    img_height, img_width, img_channels = image.shape

    resized = np.zeros((height, width, img_channels))

    x_ratio = float(img_width - 1) / (width - 1) if width > 1 else 0
    y_ratio = float(img_height - 1) / (height - 1) if height > 1 else 0

    for i in range(height):
        for j in range(width):
            x_l, y_l = math.floor(x_ratio * j), math.floor(y_ratio * i)
            x_h, y_h = math.ceil(x_ratio * j), math.ceil(y_ratio * i)

            x_weight = (x_ratio * j) - x_l
            y_weight = (y_ratio * i) - y_l

            a = image[y_l, x_l, :]
            b = image[y_l, x_h, :]
            c = image[y_h, x_l, :]
            d = image[y_h, x_h, :]

            pixel = a * (1 - x_weight) * (1 - y_weight)\
                + b * x_weight * (1 - y_weight)\
                + c * y_weight * (1 - x_weight)\
                + d * x_weight * y_weight

            resized[i, j, :] = pixel

    return resized.astype(np.uint8)

def bilinear_resize_vectorized(image, height, width):
    """
    Parameters
    ----------
    image : 2D numpy array.
    height : desired height
    width : desired width

    Returns
    -------
    resized image

    """
    img_height, img_width, img_channels = image.shape
    resized_img = []

    print(img_height)
    print(img_width)
    for c in range(img_channels):
        img_c = image[:, :, c].ravel()

        x_ratio = float(img_width - 1) / (width - 1) if width > 1 else 0
        y_ratio = float(img_height - 1) / (height - 1) if height > 1 else 0

        y, x = np.divmod(np.arange(height * width), width)

        x_l = np.floor(x_ratio * x).astype(np.uint32)
        y_l = np.floor(y_ratio * y).astype(np.uint32)
```

```
        x_h = np.ceil(x_ratio * x).astype(np.uint32)
        y_h = np.ceil(y_ratio * y).astype(np.uint32)

        x_weight = (x_ratio * x) - x_l
        y_weight = (y_ratio * y) - y_l

        a = img_c[y_l * img_width + x_l]
        b = img_c[y_l * img_width + x_h]
        c = img_c[y_h * img_width + x_l]
        d = img_c[y_h * img_width + x_h]

        resized = a * (1 - x_weight) * (1 - y_weight) + \
                  b * x_weight * (1 - y_weight) + \
                  c * y_weight * (1 - x_weight) + \
                  d * x_weight * y_weight
        resized_img.append(resized.reshape(height, width).astype(np.uint8))

    return np.stack(resized_img, axis=2)

def bilinear_resize_vectorized_backward(image, height, width, grad):
    """
    `image` is a 2-D array, holding the input image
    `height` and `width` are the desired spatial dimension of the new 2-D array.
    `grad` is a 2-D array of shape [height, width], holding the gradient to be
    backpropped to `image`.
    """
    # This function implements backward for a single channel only
    print(image.shape)
    img_height, img_width = image.shape
    image = image.ravel()
    x_ratio = float(img_width - 1) / (width - 1) if width > 1 else 0
    y_ratio = float(img_height - 1) / (height - 1) if height > 1 else 0
    y, x = np.divmod(np.arange(height * width), width)
    x_l = np.floor(x_ratio * x).astype('int32')
    y_l = np.floor(y_ratio * y).astype('int32')
    x_h = np.ceil(x_ratio * x).astype('int32')
    y_h = np.ceil(y_ratio * y).astype('int32')
    x_weight = (x_ratio * x) - x_l
    y_weight = (y_ratio * y) - y_l
    grad = grad.ravel()
    # gradient wrt `a`, `b`, `c`, `d`
    d_a = (1 - x_weight) * (1 - y_weight) * grad
    d_b = x_weight * (1 - y_weight) * grad
    d_c = y_weight * (1 - x_weight) * grad
    d_d = x_weight * y_weight * grad
    # [4 * height * width]
    grad = np.concatenate([d_a, d_b, d_c, d_d])
    # [4 * height * width]
    indices = np.concatenate([y_l * img_width + x_l,
    y_l * img_width + x_h,
    y_h * img_width + x_l,
    y_h * img_width + x_h])
    # we must route gradients in `grad` to the correct indices of `image` in
    # `indices`, e.g. only entries of indices `y_l * img_width + x_l` in `image`
    # gets the gradient backpropped from `a`.
    # use numpy's broadcasting rule to generate 2-D array of shape
    # [4 * height * width, img_height * img_width]
    indices = (indices.reshape((-1, 1)) ==
```

```
    np.arange(img_height * img_width).reshape((1, -1)))
    d_image = np.apply_along_axis(lambda col: grad[col].sum(), 0, indices)
    return d_image.reshape((img_height, img_width))

if __name__ == '__main__':
    # 1D example
    in_array = [1,2,3,4]
    out_array_smaller = linear_resize(in_array, 2)
    print(out_array_smaller)
    out_array_larger = linear_resize(in_array, 7)
    print(out_array_larger)

    # Load image
    img=cv2.imread('bender.png')
    print(img.shape)
    cv2.imshow('image', img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
    # Resize image
    height = 128
    width = 400
    resized_img = bilinear_resize(img, height, width)
    print(resized_img.shape)
    cv2.imshow('resized_image', resized_img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    resized_img = bilinear_resize_vectorized(img, height, width)
    print(resized_img.shape)
    cv2.imshow('resized_image', resized_img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    input_tensor = torch.randn(1, 1, 5, 1)
    input_tensor.requires_grad_(True)
    output_tensor = F.interpolate(input_tensor, size=(6, 6), align_corners=True, mode='bilinear')
    print(output_tensor.sum().backward())
    torch_grad = input_tensor.grad
    print(torch_grad)
    print(input_tensor.squeeze(0,3).shape)

    grad = torch.ones((6, 6))  # Pytorch does this internally during backward
    manual_grad = bilinear_resize_vectorized_backward(input_tensor.squeeze(0,3).detach().cpu().numpy(), 6,
6, grad.detach().cpu().numpy())
    print(manual_grad)
```

Related:

1. Spatial Transformer network
2. Fully Convolutional Network (FCN)

References:

1. https://chao-ji.github.io/jekyll/update/2018/07/19/BilinearResize.html